Research Activity: Alternating Least Squares (ALS)

Student Name: Farah Abdelaziz Abdelnaby
Student ID: 223102501

Abstract

Recommender systems help users discover relevant items from large collections. In collaborative filtering, recommendations are inferred from patterns in user–item interactions. Alternating Least Squares (ALS) is a matrix-factorization method that represents users and items with compact latent-factor vectors. It alternately updates user factors while item factors are fixed, then updates item factors while user factors are fixed. This report explains the main concepts behind ALS, distinguishes explicit from implicit feedback, discusses important hyperparameters and limitations, and proposes a reproducible implementation plan for explicit movie ratings. The plan includes data preparation, leakage-free splitting, model selection, alternating updates, regularization, prediction, evaluation, and cold-start handling. It is a design proposal only; no fabricated performance results are included.

1. Introduction

A user–item rating matrix records how users rate items such as movies. In a real dataset, most users rate only a small fraction of all available movies, so the matrix is sparse. A recommender attempts to estimate ratings for unobserved user–item pairs and rank items that a user may like. Collaborative filtering uses interaction patterns across users and items rather than requiring a detailed description of every item.

Matrix factorization provides a compact representation of this interaction matrix. ALS is useful because each half-step becomes a least-squares optimization problem when the other factor matrix is fixed. This structure supports iterative optimization and can be implemented in distributed systems. The method does not guarantee that every recommendation is correct; its quality depends on data coverage, model settings, and evaluation.

The report follows the assignment structure: Section A answers the eight foundational research questions, and Section B presents a ten-step plan for explicit-feedback ALS.

2. Section A — Foundational Research Questions

Q1. What does ALS stand for, and what problem does it solve?

ALS stands for Alternating Least Squares. In collaborative filtering, it learns user and item latent factors from observed interactions so that the model can estimate missing ratings. For example, a model trained on movie ratings can estimate how much a user might rate a movie they have not rated. ALS is an optimization method for learning the factors; it is not itself a guarantee that the recommendations will be useful.

Q2. What is matrix factorization? Explain factors and the dimension k.

Let R be an m × n user–item matrix, where m is the number of users and n is the number of items. Matrix factorization approximates R using two lower-dimensional factor matrices: U with shape m × k and V with shape n × k. The predicted rating for user u and item i is commonly calculated as the dot product of their vectors: $\hat{r}_{ui} = U_u \cdot V_i$. The value k is the number of latent factors. These factors represent learned patterns in interactions; they do not necessarily correspond to directly named concepts such as genre.

For example, if m = 5,000 users, n = 100 movies, and k = 20, U has shape 5,000 × 20 and V has shape 100 × 20. The approximation $U V^T$ has shape 5,000 × 100. Only observed ratings are used as rating targets; a missing entry is not automatically a rating of zero.

Q3. Why is the method called alternating?

The method alternates between optimizing U and V. First, V is held fixed and the best user vectors are estimated by solving least-squares problems. Next, U is held fixed and the item vectors are updated. These two stages are repeated for a chosen number of iterations or until a stopping condition is met. With one factor matrix fixed, the objective is quadratic in the other matrix, making the corresponding update a least-squares problem. The full joint optimization is not generally solved in one step.

Q4. Explain the loss function, regularization, and observed ratings.

For explicit ratings, a common objective minimizes squared prediction error over the observed user–item pairs. A regularization term penalizes excessively large factor values, helping reduce overfitting. In simplified form, the objective is the sum of $(r_{ui} - U_u \cdot V_i)^2$ over observed ratings plus λ times a penalty on the factor magnitudes. Here, λ controls regularization strength. Missing entries must not be blindly treated as zero because the user may never have rated the item; zero would incorrectly turn unknown preference into a numerical target. The training objective and evaluation must therefore use observed ratings and a clearly defined split.

Q5. When is ALS useful, and how does it compare with neighborhood methods?

ALS is useful when a dataset contains many user–item interactions and a compact latent representation is desirable. Factorization can capture indirect patterns across many users and items, and alternating updates can be parallelized. Neighborhood-based collaborative filtering instead finds similar users or similar items from their rating patterns and predicts from neighbors. Neighborhood methods can be easier to explain and implement for small datasets, but similarity calculations and sparse overlap can become limiting at scale. ALS can scale well and generalize through latent factors, but requires parameter tuning and is less directly interpretable. The better approach depends on the data and evaluation results.

Section A — Continued

Q6. Distinguish explicit and implicit feedback.

Explicit feedback is a direct preference signal, such as a user assigning a movie 1–5 stars. Implicit feedback is inferred from behavior, such as clicks, views, watch time, or purchases. A click does not necessarily mean that the user liked an item, and the absence of a click does not prove dislike. Explicit-feedback ALS models the numerical ratings directly on observed pairs. Implicit-feedback ALS instead commonly models a preference signal and a confidence value derived from behavior. Therefore, implicit ALS uses a different objective and must not be treated as ordinary 1–5 rating prediction.

Q7. Which hyperparameters affect quality?

The main hyperparameters include rank k, regularization strength λ, and the number of iterations. A very small rank may underfit because the model cannot represent enough interaction patterns. A very large rank can increase computation and overfit sparse data. Too little regularization may overfit observed ratings; too much may oversimplify the model. Too few iterations may stop before useful convergence, while excessive iterations increase runtime and may provide little improvement. Initialization, random seed, and implementation-specific settings can also affect reproducibility or performance. Values should be selected using validation data rather than chosen from test-set performance.

Q8. Discuss cold start, popularity imbalance, and sparsity.

ALS has a cold-start limitation for a completely new user or item with no interactions, because the model has no observed ratings from which to learn its latent vector. Sparse users and items also have less evidence, so their factor estimates may be less reliable. Popularity imbalance can make recommendations favor frequently rated items and under-serve niche interests. A practical fallback for a new user is to recommend globally popular or highly rated items, ask the user to rate a small set of items, or use content-based information if available. For a new item, metadata-based recommendations or a temporary popularity/exploration strategy can be used until enough interactions are collected. These fallbacks should be evaluated separately from warm-start ALS predictions.

3. Section B — Explicit-Feedback ALS Implementation Plan

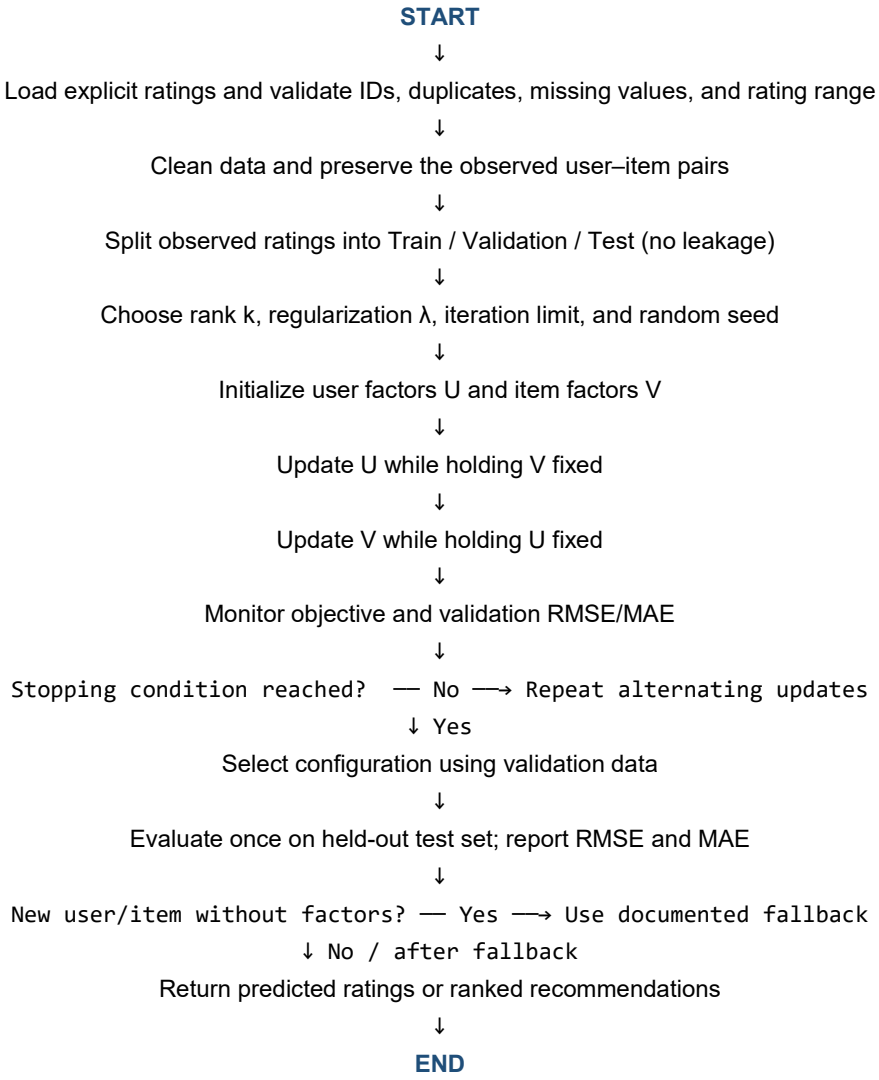
Goal: design a reproducible ALS pipeline for a dataset containing explicit 1–5 movie ratings. This is an algorithm plan, not a claim that a model has been implemented or evaluated.

Step / Purpose	Input	Expected output	Decision / check
1. Load and validate data	Read user_id, item_id, and rating; check data types, missing IDs, duplicate user–item pairs, and ratings outside 1–5.	A clean interaction table and an integrity report.	If duplicates exist, investigate the source and define a documented policy; do not silently discard records.
2. Map identifiers and define observations	Create consistent integer mappings if required by the implementation; retain a mapping back to original IDs.	A valid user–item–rating table and explicit list of observed pairs.	Never convert unobserved pairs to rating zero.
3. Split the data without leakage	Split observed interactions into training, validation, and test sets, for example 80/10/10 with a fixed seed.	Three disjoint sets of observed ratings.	Check that no rating record appears in more than one split; use a protocol appropriate to the intended task.
4. Set baseline and metrics	Calculate a simple training-mean baseline and define RMSE and MAE before tuning.	Baseline predictions and a fixed evaluation protocol.	Ensure all comparisons use the same held-out test records and report coverage.
5. Choose factorization parameters	Choose candidate rank k, regularization λ, iteration limit, and a reproducible random seed.	A documented search space and initial configuration.	Select values using validation performance, not test results.
6. Initialize factor matrices	Initialize user factors U (m × k) and item factors V (n × k), using the implementation's supported initialization.	Initial latent-factor matrices.	Check dimensions, finite values, and correct user/item index alignment.
7. Alternate least-squares updates	With V fixed, solve for user vectors U using training ratings;	Updated U and V after each iteration.	Use only training interactions during fitting; apply regularization in each

	then hold U fixed and solve for V.		update.
8. Monitor and stop training	Track training objective or validation RMSE/MAE across iterations; use a maximum iteration count and optional early stopping.	Selected model checkpoint and training log.	Stop when the iteration limit is reached or validation improvement stalls according to a predefined rule.
9. Tune and select the model	Compare candidate ranks and regularization strengths using validation metrics.	The configuration with the best validation result under the chosen criterion.	Avoid repeatedly selecting settings based on test-set scores.
10. Test, predict, and handle cold start	Predict held-out ratings using user/item vector dot products; compute final RMSE and MAE; define fallback recommendations.	Final test metrics, example predictions, and a cold-start policy.	Report results only after the model is actually run; route users/items with no learned factors to the fallback.

4. Flowchart of the Proposed ALS Pipeline

The flow below summarizes the proposed algorithm. It describes the intended process and does not represent a completed experiment.



5. Evaluation Protocol and Expected Checks

RMSE (root mean squared error) penalizes larger prediction errors more strongly, while MAE (mean absolute error) reports the average absolute difference between true and predicted ratings. Lower values indicate smaller errors on the same evaluation set, but they do not by themselves prove that a recommender produces better-ranked recommendations. The report should state the split strategy, random seed, hyperparameters, number of evaluated ratings, and treatment of cold-start cases. No numeric result should be inserted until the algorithm is implemented and run.

A reliable evaluation avoids data leakage: ratings in validation and test sets must not influence factor learning or hyperparameter selection. For a random interaction split, some users/items may be absent from the training set; the implementation must either exclude these cases from warm-start metrics with a clear coverage report or evaluate a predefined fallback separately. If the goal is future recommendation, a time-based split may be more appropriate than a random split.

6. Conclusion

ALS is a matrix-factorization approach for collaborative filtering that learns compact user and item vectors by alternating least-squares updates. It can be effective for sparse rating data, but it depends on adequate interaction coverage, regularization, suitable hyperparameters, and careful evaluation. Explicit ratings should be modeled as observed numerical preferences, whereas implicit actions require a different interpretation. The proposed ten-step plan provides a reproducible path from data validation to evaluation and cold-start handling. Actual model quality must be established through experiments rather than assumed.

References

[1] Y. Hu, Y. Koren, and C. Volinsky, “Collaborative Filtering for Implicit Feedback Datasets,” in Proceedings of the 2008 IEEE International Conference on Data Mining (ICDM), 2008, pp. 263–272. https://doi.org/10.1109/ICDM.2008.22

[2] Apache Spark, “Collaborative Filtering,” Spark MLlib Documentation. Available: https://spark.apache.org/docs/latest/ml-collaborative-filtering.html

[3] Apache Spark, “pyspark.ml.recommendation.ALS,” PySpark API Documentation. Available: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.recommendation.ALS.html